

Apuntes Semana 11 [Martes]

Curso: IC6200 Inteligencia Artificial

Clase del 05 de mayo de 2026

Jose David Chaves Mena - 2024072126

Tecnologico de Costa Rica
Escuela de Ingenieria en Computacion
Profesor: Steven Pacheco Portuguez
Fecha: 05 de mayo de 2026

Abstract

Este documento presenta los conceptos fundamentales de las Redes Neuronales Convolucionales (CNN), incluyendo capas convolucionales, pooling, pesos compartidos y capas totalmente conectadas. Además, se describen arquitecturas importantes como LeNet, AlexNet, GoogleNet, VGG16 y ResNet, junto con sus principales características, ventajas y limitaciones.

1. Discusión de resultados del proyecto

En la discusión previa a la clase, la mayoría de los grupos indicó haber utilizado *MobileNetV2*. En general, se mencionó que este modelo fue una de las mejores opciones para el proyecto debido a su buen equilibrio entre rendimiento y cantidad de parámetros.

De forma aproximada, varios grupos reportaron una exactitud cercana a 0.8 con la opción B, mientras que con la opción A se alcanzaron resultados cercanos a 0.9 en algunos casos. En la mayoría de las intervenciones, *MobileNet* fue la arquitectura más utilizada, principalmente por su eficiencia computacional y su buen desempeño en tareas de clasificación.

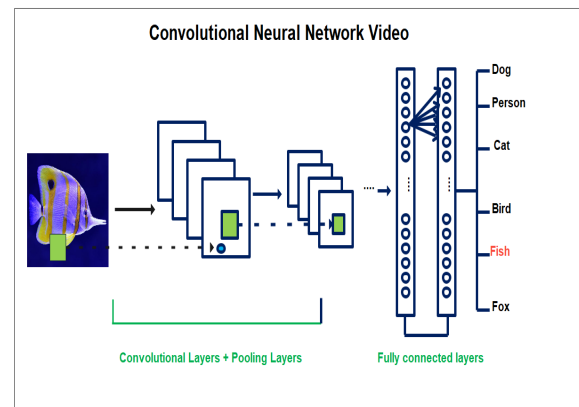


Figure 1: Flujo general del procesamiento en una red neuronal convolucional.

2. Arquitectura de una CNN

Una **red neuronal convolucional** (*Convolutional Neural Network*, CNN) es una arquitectura diseñada para procesar imágenes y extraer características relevantes de forma progresiva.

2.1. Capa de convolución

La capa de convolución permite extraer *features* o características importantes de la imagen, como bordes, texturas, formas y patrones más complejos. El proceso consiste en aplicar filtros o *kernels* sobre regiones pequeñas de la imagen para detectar patrones locales.

A medida que la información avanza por la red, la representación de la imagen se va condensando. Primero se extraen características básicas, luego patrones más complejos y, finalmente, se obtiene una representación compacta en forma de vector que resume la información más importante de la imagen.

Por ejemplo, en la arquitectura de **LeNet** se observan varias etapas de convolución seguidas por una capa totalmente conectada. La imagen mostrada anteriormente corresponde a un ejemplo de arquitectura como **AlexNet**.

2.2. Kernels y mapas de características

Los **kernels** son pequeños filtros que recorren regiones específicas de la imagen para detectar patrones locales.

Los **mapas de características** (*feature maps*) son los resultados producidos por esos filtros. Dependiendo del diseño de la red, se pueden escoger distintos mapas para conservar las características más relevantes durante el procesamiento.

3. Pesos compartidos

Para mejorar la eficiencia, las CNN no aprenden un conjunto distinto de pesos para cada posición de la imagen. En su lugar, utilizan **pesos compartidos**: el mismo filtro se aplica a toda la imagen.

Esto significa que si un extractor de características funciona para una posición (x, y) , también puede aplicarse en $(x_1, y_1), (x_2, y_2), \dots, (x_n, y_n)$. Esta propiedad reduce con-

siderablemente la cantidad de parámetros que debe aprender la red.

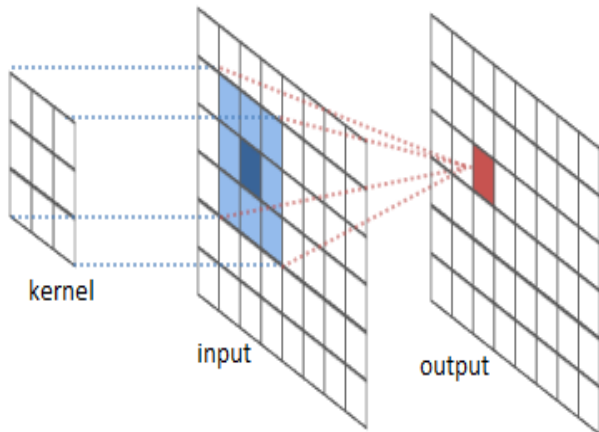


Figure 2: Ejemplo de kernel y pesos compartidos en una CNN.

3.1. Ejemplo: 96 filtros en AlexNet

En AlexNet, cada filtro procesa una entrada de $11 \times 11 \times 3$ y la capa de salida produce $55 \times 55 \times 96$, como resultado de aplicar los 96 filtros a lo largo de una imagen de entrada de $227 \times 227 \times 3$.

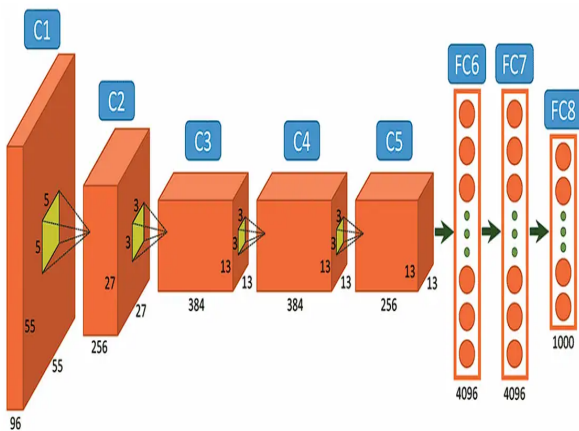


Figure 3: Filtros y mapas de características en AlexNet.

En esta imagen se puede observar cómo cada mapa de características ayuda a identificar bordes horizontales, diagonales, colores y otros patrones relevantes.

4. Pooling

El **pooling** es una operación utilizada para reducir las dimensiones espaciales de los mapas de activación, disminuyendo la cantidad de información que procesa la red. Generalmente se aplica después de una capa convolucional y de una función de activación.

Su objetivo principal es reducir el costo computacional y la cantidad de parámetros sin perder las características más importantes. En la práctica, se utiliza con frecuencia **max pooling**, normalmente con un *stride* de 2 y un filtro de 2×2 .

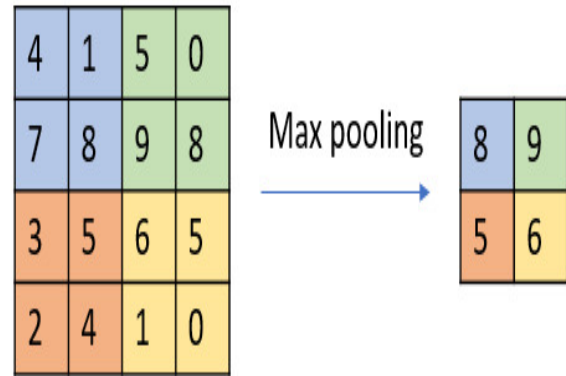


Figure 4: Ejemplo de max pooling con filtro 2×2 .

En este método se toma una región pequeña del mapa de activación, se selecciona el valor máximo y ese valor representa toda la región.

4.1. Dimensionalidad del pooling

Dada una entrada de ancho W , alto H y profundidad D , con un kernel de tamaño k y un *stride* S , la dimensión de salida se calcula como:

$$W_{out} = \frac{W - k}{S} + 1$$

De forma análoga, se calcula el alto de salida. La profundidad, normalmente, se mantiene constante.

Otras técnicas como **average pooling** y **L2 pooling** también existen, aunque en muchos casos **max pooling** ofrece mejores resultados prácticos.

La idea es que, después de las capas de convolución y pooling, se obtenga una representación de la imagen que pueda convertirse en un vector para la siguiente etapa: la capa totalmente conectada.

5. Capa totalmente conectada

La **fully connected layer** funciona de manera similar a una red neuronal tradicional. Utiliza las características extraídas previamente para realizar la clasificación final.

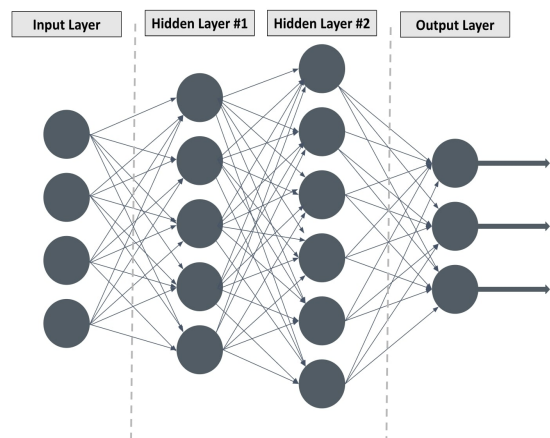


Figure 5: Resumen del proceso: convolución, extracción de características y capa totalmente conectada.

Cuanto mayor sea la capacidad del vector de entrada para esta capa, mejor puede ser el desempeño del modelo. Sin embargo, también aumenta el costo computacional, por lo que es importante encontrar un equilibrio entre la cantidad de parámetros y la calidad del resultado.

6. Arquitecturas convolucionales: reglas, estándares y recomendaciones

6.1. Convoluciones pequeñas

Conviene utilizar convoluciones pequeñas, por ejemplo de 3×3 o 5×5 .

Por ejemplo, una secuencia de tres convoluciones de 3×3 puede lograr un campo receptivo equivalente al de una convolución de 7×7 .

La primera convolución observa una región de 3×3 de la imagen original. La segunda, al conectarse con la salida anterior, observa un área efectiva de 5×5 . Finalmente, la tercera permite observar un área total de 7×7 .

Aunque el resultado espacial es similar al de una sola convolución de 7×7 , el uso de bloques pequeños presenta ventajas importantes:

- menor cantidad de parámetros,
- menor costo computacional,
- más funciones no lineales, como ReLU,
- características más expresivas.

Suponiendo que los volúmenes poseen C canales, una convolución de 7×7 requiere:

$$(7 \times 7 \times C) \times C = 49C^2$$

Por otro lado, un bloque de tres convoluciones de 3×3 requiere:

$$3 \times (3 \times 3 \times C) \times C = 27C^2$$

Esto representa una reducción significativa en el número de parámetros.

6.2. Otras recomendaciones

- El tamaño de la capa de entrada suele ser divisible entre 2 varias veces, para facilitar la reducción progresiva de dimensiones.
- En pooling, la configuración más común es un filtro de tamaño 2 con *stride* 2; también es frecuente usar filtros de 3×3 con *stride* 2.

7. Ejemplos de arquitecturas

7.1. LeNet

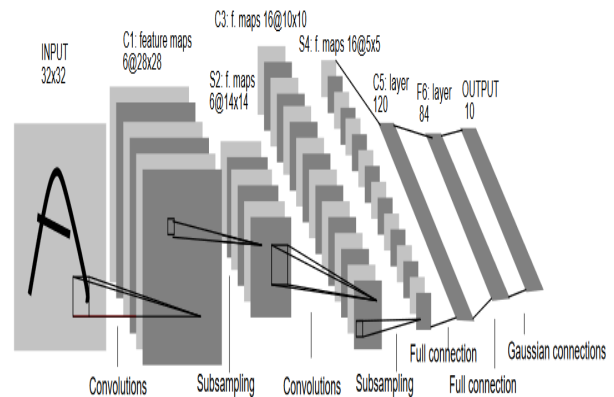


Figure 6: Arquitectura de LeNet.

7.2. AlexNet

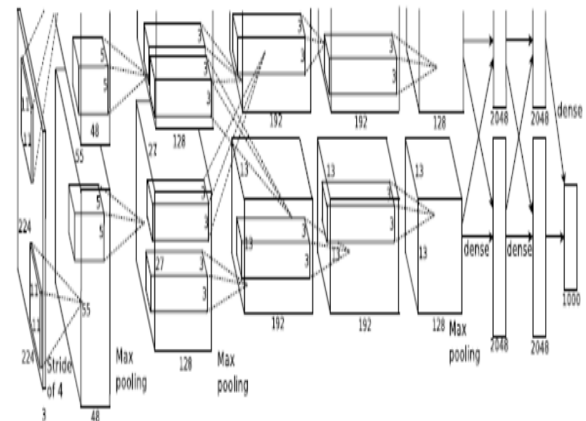


Figure 7: Arquitectura de AlexNet.

7.3. ZFNet

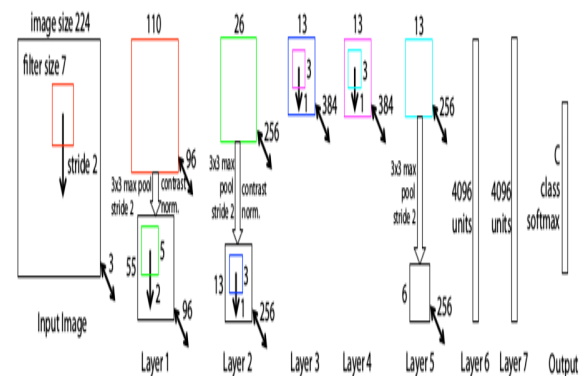


Figure 8: Arquitectura de ZFNet.

ZFNet es similar a AlexNet, pero utiliza kernels de menor tamaño y ajustes que mejoran la visualización e interpretación de los filtros.

7.4. GoogleNet

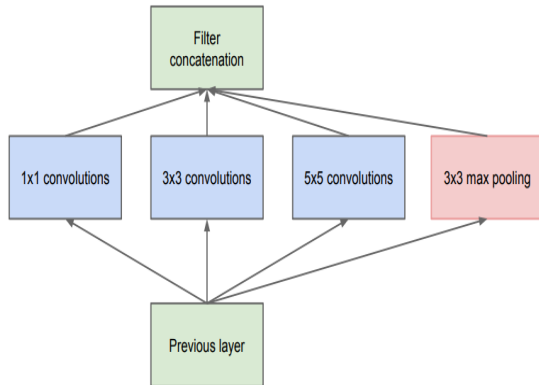


Figure 9: Arquitectura de GoogleNet.

GoogleNet permitió una reducción importante de parámetros, pasando aproximadamente de 60 millones en AlexNet a alrededor de 4 millones en su diseño. Esto permite:

- capturar información a distintas escalas,
- detectar patrones de diferentes tamaños,
- mejorar el rendimiento con menos parámetros.

En consecuencia, la arquitectura mejoró de forma considerable la eficiencia computacional de las CNN profundas.

7.5. VGG16

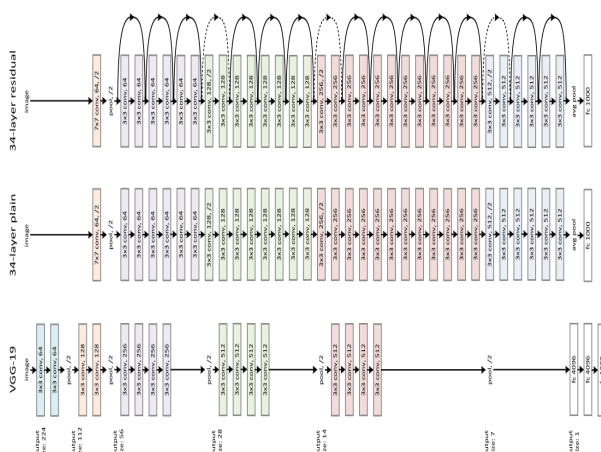


Figure 10: Arquitectura de VGG16 y ResNet.

VGG16 tiene 16 capas profundas y demostró que aumentar la profundidad de una red puede mejorar significativamente el aprendizaje de características complejas. Entre sus principales características destacan:

- uso de convoluciones pequeñas,

- arquitectura uniforme,
- gran profundidad.

7.6. ResNet

ResNet introdujo el concepto de **conexiones residuales** (*skip connections*), lo que permitió entrenar redes neuronales extremadamente profundas. Un ejemplo es una arquitectura con 34 capas.

8. Limitaciones y problemas de las CNN

Uno de los principales problemas de las CNN es su **baja explicabilidad**. En muchos casos, estas redes funcionan como una *caja negra*, lo que dificulta entender con precisión cómo toman decisiones.

Además, los *features* aprendidos no siempre son fáciles de interpretar, especialmente cuando la red tiene muchas capas y aprende representaciones muy abstractas.